LangChain이란

2026.3.16

장신원

목 차

1. Background & Why

- LLM의 텍스트 이해 : Attention
- LLM의 텍스트 생성 : AutoRegressive
- LLM은 결국..?
- Why LangChain?

2. What

- Runnable
- Pipeline I/O
- PromptTemplate
- Models

3. How

- LangChain Expression Language

4. Q&A

Background
&
Why

LLM 작동방식 & 원리(Simple)

- LLM의 텍스트 이해(Attention)

신원은 개를 키운다. 그의 개가 다른 사람의 개를 물었다.

대명사 '그'가 가리키는 것이 무엇인지 알지 못함!

수학적으로 각각의 '개'는 동일한 벡터!

=> Attention은 QKV 행렬과 내적을 이용하여 해결!

∴ Attention은 LLM에게 텍스트 이해력을 부여한다

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

내적 값의 해석

양수 : 관련성 있음

0 : 관련 없음

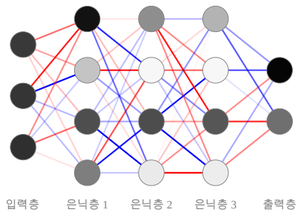
음수 : 반대의 의미

LLM 작동방식 & 원리(Simple)

– LLM의 텍스트 생성(AutoRegressive)

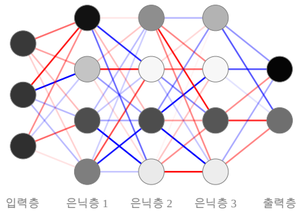
입력 | 출력

"나는"



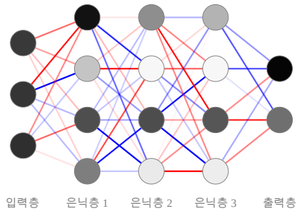
"너를" ··· 80%
"사과를" ··· 10%
"공부를"··· 5%
⋮

"나는 너를"



"좋아한다" ··· 70%
"싫어한다" ··· 20%
"밥이"··· 5%
⋮

"나는 너를 좋아한다"



"<EOS>" ··· 85%
"그런데" ··· 15%
"자동차가"··· 3%
⋮

"나는 너를 좋아한다 <EOS>"

∴ AutoRegressive는 LLM 에게 텍스트 생성력을 부여한다.

LLM 작동방식 & 원리(Simple)

- LLM은 결국..?

오늘은 구름이 많네, 오늘 날씨가 어떻게 될까?

LLM : (구름이 많아..? 비가 오겠군) 비가 올 것 같습니다.

LLM 작동방식 & 원리(Simple)

- LLM은 결국..?

- 정보의 단절(Cut-off) : 모델의 가중치는 자신이 학습한 데이터에 한정된다. 최신 정보는 반영하지 못한다.
- 행동 능력의 부재 : LLM은 텍스트만 생성할 수 있을 뿐, 실제로 DB에 쿼리를 날리거나 웹에 검색할 수 있는 능력이 없다.
- Stateless : LLM은 매번 단어 확률을 계산할 뿐, 대화 맥락을 기억 하지 못한다.

∴ Hallucination 심화

LLM 작동방식 & 원리(Simple)

– Why Langchain?

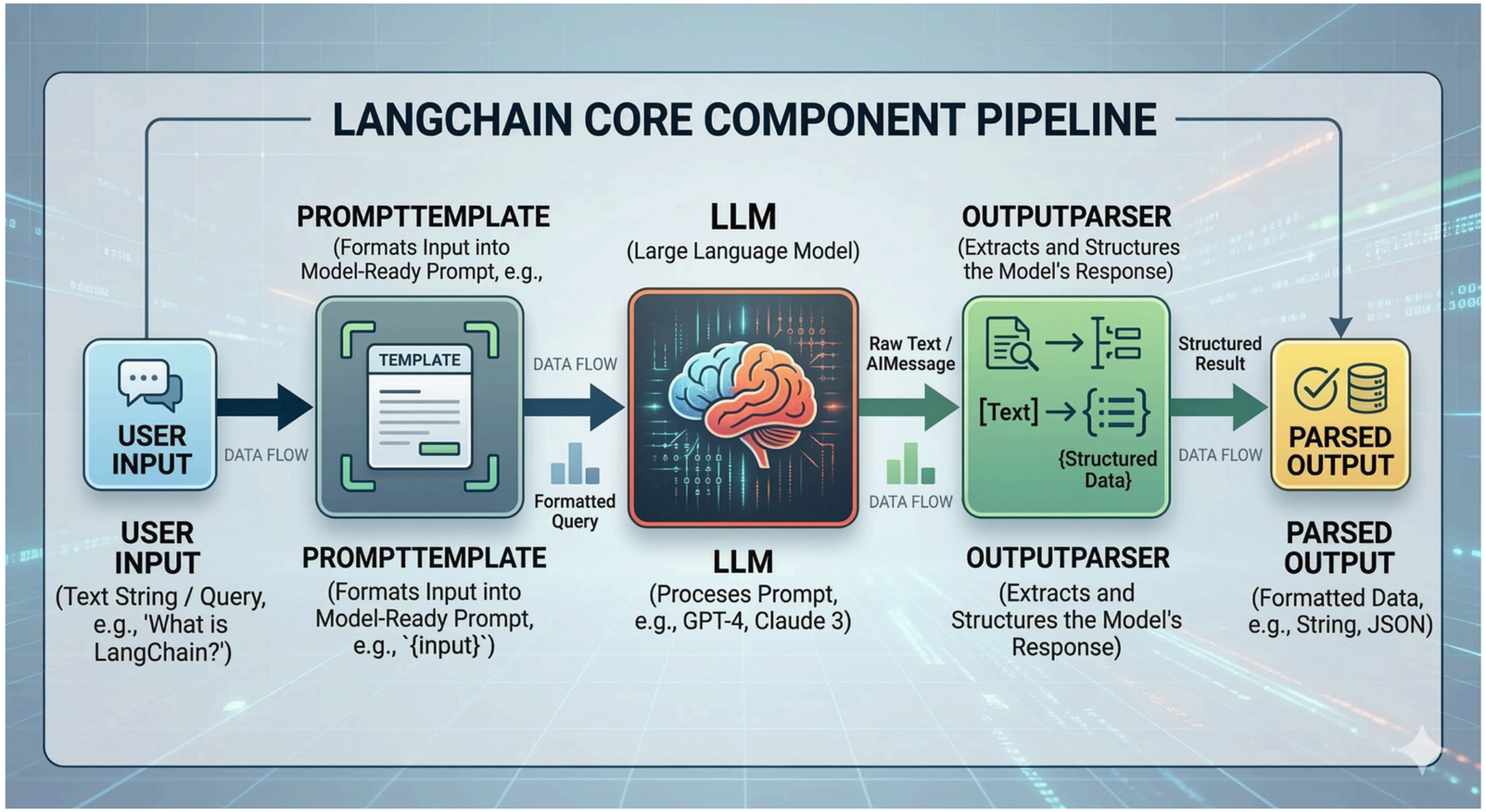
- RAG(Retrieval Augmented Generation) & Agent
- 표준화된 인터페이스 & 방대한 생태계 지원
- LLM을 추론 엔진으로 활용한 애플리케이션

LangChain Framework

- Prompt
- Retrieval
- Memory
- Chain

⋮

What


LANGCHAIN CORE COMPONENT PIPELINE
PROMPTTEMPLATE
(Formats Input into Model-Ready Prompt, e.g.,
LLM
(Large Language Model)
OUTPUTPARSER
(Extracts and Structures the Model's Response)
USER INPUT
DATA FLOW
TEMPLATE
DATA FLOW
Formatted Query
Raw Text / AIMessage
DATA FLOW
[Text]
{Structured Data}
Structured Result
DATA FLOW
PARSED OUTPUT
USER INPUT
(Text String / Query, e.g., 'What is LangChain?')
PROMPTTEMPLATE
(Formats Input into Model-Ready Prompt, e.g., `{input}`)
LLM
(Proceses Prompt, e.g., GPT-4, Claude 3)
OUTPUTPARSER
(Extracts and Structures the Model's Response)
PARSED OUTPUT
(Formatted Data, e.g., String, JSON)

What?

- Runnable Interface

ChatOpenAI
Prompt
OutputParser
⋮

Standard Interface
- invoke()
- stream()
- batch()

+

비동기 Interface
- ainvoke()
- astream()
- abatch()

=

다형성
(Polymorphism)
&
체인의 손쉬운 사용

What?

- Pipeline Input/Output

{“question”: “LCEL이 뭐야?”} ·····Dict

↓

[(“system”, “당신은 유능한 AI 비서입니다. 사용자의 question에 친절히 대답해주세요”),
(“human”, “{question}”)]

····· ChatPromptValue

↓

{“LLM” : LCEL이란 ..., “metadata”: ... ,} ·····AIMessage

↓

“LCEL이란 ...” ·····String

What?

– OutputParser와 타입 안정성

```python
from langchain_core.output_parsers import CommaSeparatedListOutputParser

parser = CommaSeparatedListOutputParser()
print(parser.get_format_instructions())
```

✓ 0.0s

```
Your response should be a list of comma separated values, eg: `foo, bar, baz` or `foo,bar,baz`
```

기존 OutputParser : LLM에게 '부탁'하는 방식

```python
class appointment(BaseModel):
    date: str = Field(description="약속의 날짜 입니다.  YYYY-MM-DD 형식이어야 합니다.")
    time: str = Field(description="약속의 시간 입니다. HH:MM format 형식이어야 합니다.")
    location: str = Field(description="약속의 장소 입니다.")
    description: str = Field(description="약속에 대한 간단한 설명입니다.")


text = {"text": "2026년 3월 16일 오후 3시에 서울에서 친구와 만나기로 했어."}

llm = ChatOpenAI(model="gpt-4o").with_structured_output(appointment)


prompt = ChatPromptTemplate(
    messages=[
        ("system", "당신은 유능한 AI 어시스턴트 입니다.."),
        ("user", "text에서 약속에 대한 정보를 추출해줘. text: {text}")
    ],
)

chain = prompt | llm
chain.invoke(text)
```

✓ 1.6s

```
appointment(date='2026-03-16', time='15:00', location='서울', description='친구와 만남')
```

with_structured_output() : native 수준에서 명령

What?

– PromptTemplate

```
2. **문서 관련성 평가:** [CONTEXT]에 포함된 각 문서를 개별적으로 평가합니다. "이 문서가 사용자의 질문에 직접적으로,그리고 의미 있게 답변하는가?"를 기준으로 '매우 관련 높음',

3. **[중요]지역 관련 질문 처리 원칙:**
   -사용자가 특정 지역(예: "제주 사는", "서울 거주")을 언급한 경우:
     * **명시적 거주지 제한이 있는 경우만**해당 지역 정책으로 판단
     *주관 기관이 해당 지역이더라도, **거주지 제한이 명시되지 않았다면** "거주지 제한 확인 필요"정책으로 분류
     *예시: "주관 기관은 제주특별자치도입니다" →제주도민 전용이 아닐 수 있음
   - **절대 추론하지 마세요:** "주관 기관이 제주니까 제주도민만 가능하겠지"와 같은 추론은 금지

4. **정보 선별 및 답변 생성:**
   - **'매우 관련 높음'**으로 판단된 문서들의 내용을 중심으로 답변을 종합합니다.
   - '약간 관련 있음'으로 판단된 문서는, '추가적으로 고려해볼 만한 정책'으로 간략하게만 언급할 수 있습니다.
   - **'관련 없음'**으로 판단된 문서는 답변 생성에 **절대 사용하지 마세요.**

5. **근거주의 원칙:**답변의 모든 내용은 반드시 선별된 [CONTEXT]문서에 명시된 사실에만 근거해야 합니다.절대 정보를 추측하거나 외부 지식을 활용하지 마세요.

6. **'정보 없음'처리:**만약 평가 결과,모든 문서가 '관련 없음'으로 판단될 경우, "죄송하지만 문의하신 조건에 정확히 부합하는 정책을 찾지 못했습니다."라고 솔직하게 답변해야 합니

7. **답변 형식:**
   -사용자가 이해하기 쉽게 친절하고 명확한 문장으로 설명합니다.
   -여러 정책을 추천할 경우,각 정책을 번호 목록으로 구분하고 **정책명**과 **핵심 지원 내용**을 명확히 요약하여 제시합니다.
   - **[중요]지역 제한 불확실성 안내:**주관 기관은 특정 지역이지만 거주지 제한이 명시되지 않은 경우,반드시 다음과 같이 안내:
     * "※ 주관 기관이 [지역명]이지만,거주지 제한이 명시되지 않아 타 지역 거주자도 지원 가능할 수 있습니다.정확한 지원 자격은 해당 기관에 문의하시기 바랍니다."
   -지원 기간과 운영 기간의 분리를 명확히 하고 둘 다 제시합니다.
   -정책과 관련된 URL정보가 있는 경우, URL을 제공하세요.

# --- [매우 중요]최종 출력 지침 (FINAL OUTPUT INSTRUCTIONS) ---
#아래 지침은 반드시 엄격하게 준수하여 최종 응답을 생성해야 합니다.
# 1. **사용자에게 직접 말하는 페르소나를 유지하세요.**절대 자신의 분석 과정("문서를 분석한 결과", "이 문서는 관련성이 높다고 판단됩니다"등)을 설명하지 마세요.
# 2. **친절한 정책 비서처럼 자연스럽게 답변을 시작하세요.**
# 3.위에서 지시된 '답변 형식'을 철저히 따라서,찾아낸 정책 정보를 명확하게 전달하는 데에만 집중하세요.
# 4. **지역 관련 불확실성이 있는 경우 반드시 안내문을 포함하세요.**

# CONTEXT
{context}

# USER'S QUERY
{question}

# RESPONSE
'''
```

1. 사용자의 입력 혹은 DB 검색 결과를 동적으로 주입
2. 관심사의 분리(SoC)
3. 역할 기반 메시지 구조를 채택함으로써 맥락 파악을 더 용이하게 수행

```python
TEMPLATE_WITH_HISTORY_FOR_R = ChatPromptTemplate.from_messages(
    [
        ("system", contextualize_q_system_prompt),
        MessagesPlaceholder("chat_history"),
        ("human", "{question}"),
    ]
)
```

What?

- Models

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

# llm = ChatOpenAI(model="gpt-4o").with_structured_output(appointment)
llm = ChatAnthropic(model="claude-2").with_structured_output(appointment)
```

OpenAI, Google, Anthropic
등의 서드파티 AI와의 API 통신을 규격화

=> 기업마다 사용 방법이 다름에도 불구하고,
객체 선언부만 교체함으로써 Model을 손쉽게 교체!

```
chain = prompt | llm
chain.invoke(text)
```

모델을 바꿨음에도 체인 선언부와 사용법은 변화 없음

How

How

- LCEL

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate(
    messages=[
        ("system", "당신은 유능한 AI 어시스턴트 입니다.."),
        ("user", "text에서 약속에 대한 정보를 추출해줘. text: {text}")
    ],
)

llm = ChatOpenAI(model="gpt-4o").with_structured_output(appointment)

text = {"text": "2026년 3월 16일 오후 3시에 서울에서 친구와 만나기로 했어."}
chain = prompt | llm
chain.invoke(text)
```

✓ 1.6s

```
appointment(date='2026-03-16', time='15:00', location='서울', description='친구와 만남')
```

Runnable 인터페이스 + ‘|’ 파이프 연산자
=> 복잡한 AI 워크 플로우에 대한
모듈화 & 강력한 캡슐화

요약

- LLM의 한계 : Attention&AutoRegressive => 확률적 텍스트 생성기
- 랭체인이란? : LLM을 논리적 추론 엔진으로 활용할 수 있도록 돕는 프레임워크
- 랭체인 구성 요소와 사용 : LCEL (Runnable + ‘|’) => LLM 애플리케이션 워크플로우의 간단한 작성

이를 활용한 대중적인 아키텍쳐는..?

Retrieval Augmented Generation

Q&A

질의 응답

Thank you!